

MODUL WEB DEVELOPMENT LARAVEL 9

STUDI KASUS SISTEM INFORMASI SEKOLAH

1. Membuat project Laravel dengan nama Laravel-sisfo

Ada beberapa cara yang dapat digunakan untuk membuat project baru Laravel yaitu dengan cara install installer Laravel secara global atau install menggunakan composer.

Global Installer Laravel

- Pertama buat folder / workspace terlebih dahulu didalam folder htdocs, kita sepakati dengan nama **Laravel**.
- Selanjutnya buka terminal / command prompt / cmd dan masuk kedalam workspace (C:\xampp\htdocs\laravel)
- Setelah masuk kedalam workspace ketikkan perintah berikut untuk melakukan instalasi.
composer global require "laravel/installer"
- Setelah instalasi selesai, selanjutnya buat project laravel dengan cara mengetikkan perintah berikut.

Laravel new Laravel-sisfo

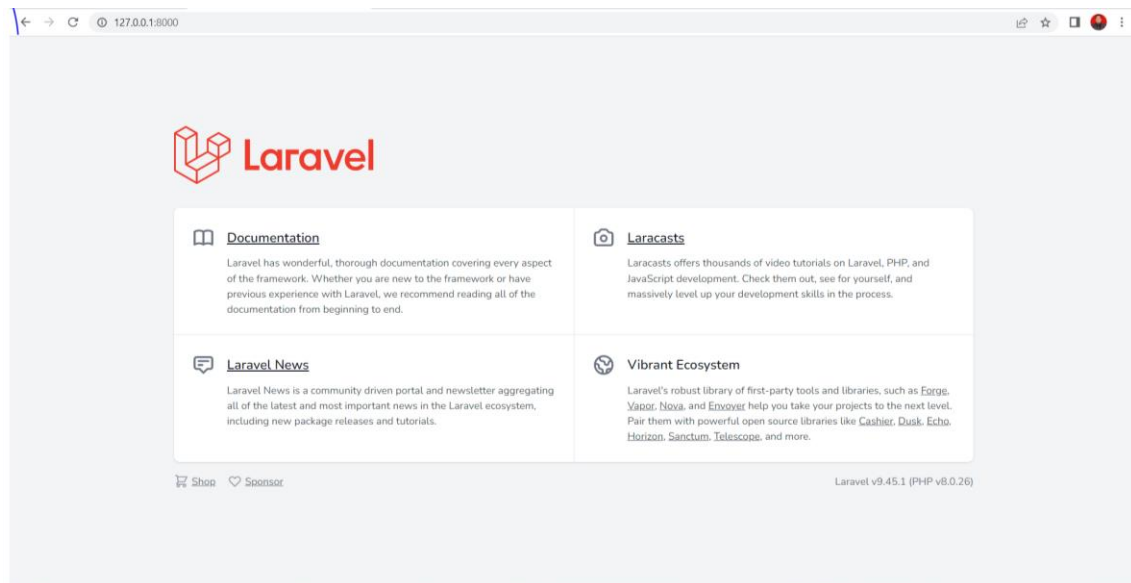
Comoposer

Cara kedua yang dapat digunakan untuk membuat project baru Laravel yaitu menggunakan comoposer, ketikkan perintah berikut pada workspace.

```
composer create-project laravel/laravel=^9.0 laravel-sisfo --prefer-dist
```

untuk menjalankan project dapat menggunakan perintah seperti dibawah ini.

php artisan serve



Gambar Tampilan Utama Larave

2. Konfigurasi Database

buka file .env kemudian isikan konfigurasi datababse berikut ini.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_sisfo
DB_USERNAME=root
DB_PASSWORD=
```

3. User Authentication

User authentication pada studi kasus ini menggunakan fitur authentication bawaan Laravel.

Install package Laravel/ui

Buka terminal/cmd kemudian ketikkan perintah berikut ini.

composer require laravel/ui

jika berhasil maka akan tampil seperti gambar berikut ini.

```
laravel/sail ..... DONE
laravel/sanctum ..... DONE
laravel/tinker ..... DONE
laravel/ui ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE
spatie/laravel-ignition ..... DONE
```

Authenctiacation fitur

Ketikkan perintah berikut pada terminal / cmd **php artisan ui bootstrap --auth**, jika berhasil maka akan tampil seperti gambar berikut.

```
PS C:\xampp\htdocs\laravel\laravel-sisfo>
>> php artisan ui bootstrap --auth

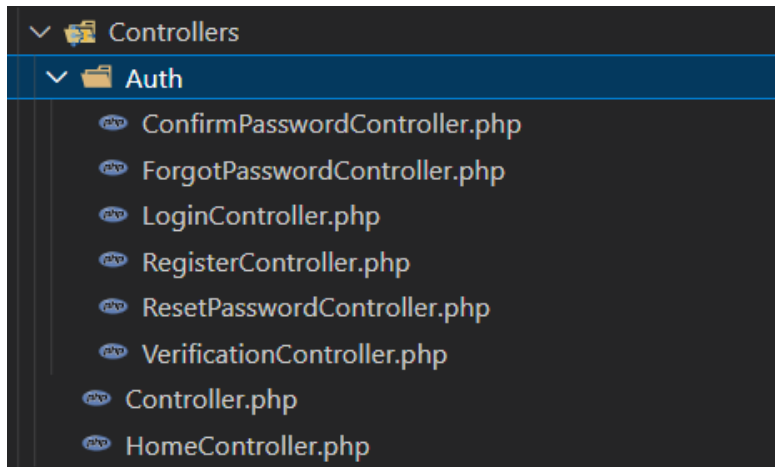
[INFO] Authentication scaffolding generated successfully.

[INFO] Bootstrap scaffolding installed successfully.

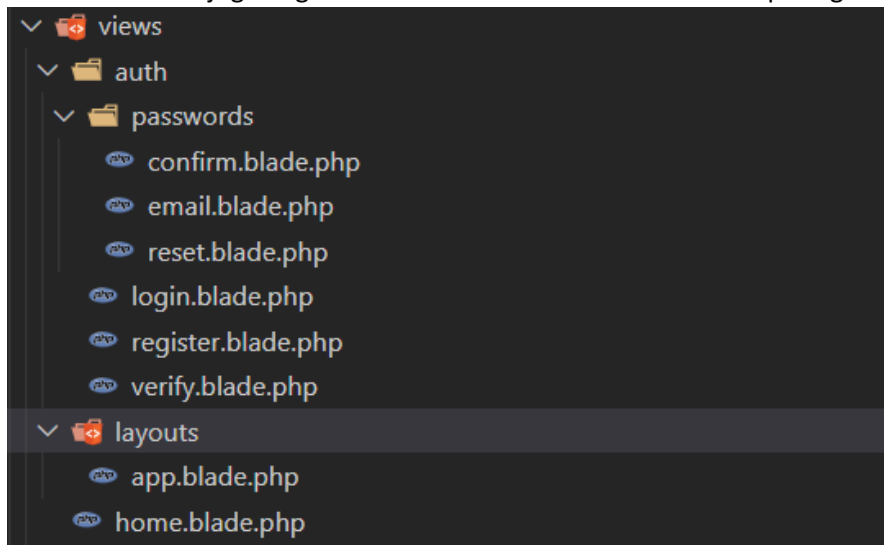
[WARN] Please run [npm install && npm run dev] to compile your fresh scaffolding.
```

Install dan compile

Ketikkan perintah **npm install && npm run dev** untuk menginstall dan compile file-file asset bawaan, dengan perintah diatas maka file-file yang dibutuhkan untuk authentication akan di generate secara otomatis, seperti halaman login, register dan forgot password, selain itu juga akan gi generate controller auth seperti gambar berikut.



Selain controller juga degenerate view untuk authentication seperti gambar berikut.



Migration

Ketikkan perintah **php artisan migrate** maka Laravel akan membuat migration table authentication, jika sebelumnya belum terdapat database maka akan ada konfirmasi dari laravel apakah akan dibuatkan databasenya atau tidak. Jika migration berhasil maka akan tampil seperti tampilan berikut.

```
C:\xampp\htdocs\laravel\laravel-sisfo
λ php artisan migrate

[WARN] The database 'laravel_sisfo' does not exist on the 'mysql' connection.

Would you like to create it? (yes/no) [no]
yes

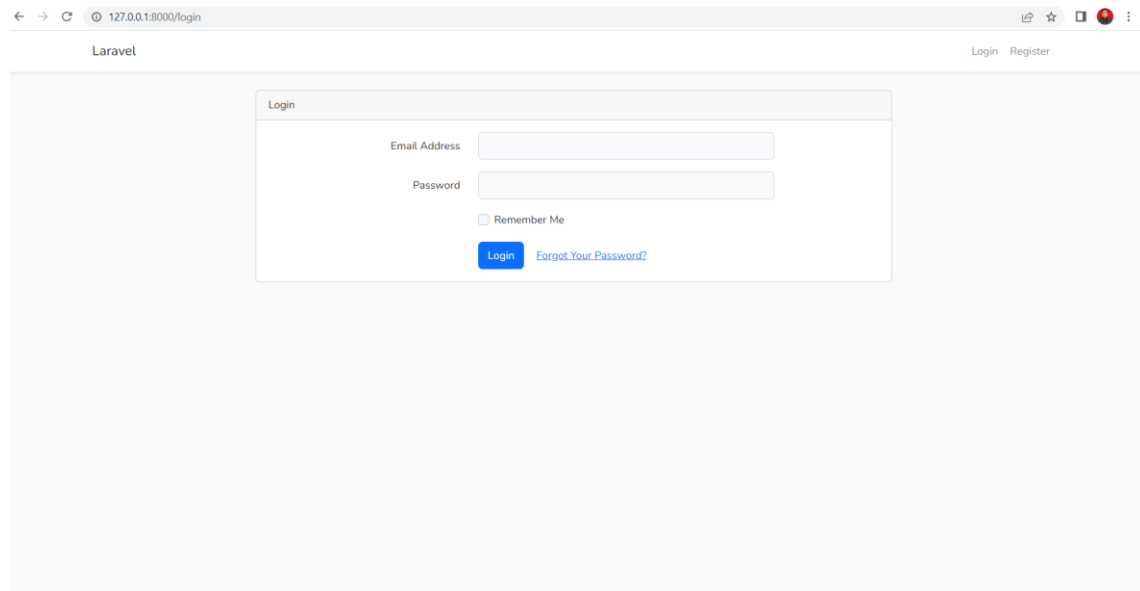
[INFO] Preparing database.

Creating migration table ..... Migration ..... 39ms DONE
[INFO] Running migrations.
2014_10_12_000000_create_users_table ..... 55ms DONE
2014_10_12_100000_create_password_resets_table ..... 37ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 41ms DONE
2019_12_14_000001_create_personal_access_tokens_table ..... 74ms DONE
```

Jika instalasi berhasil maka aplikasi sisfo sudah memiliki fitur authentication bawaan dari Laravel, berikut tampilan fitur authentication dari Laravel.

Halaman login

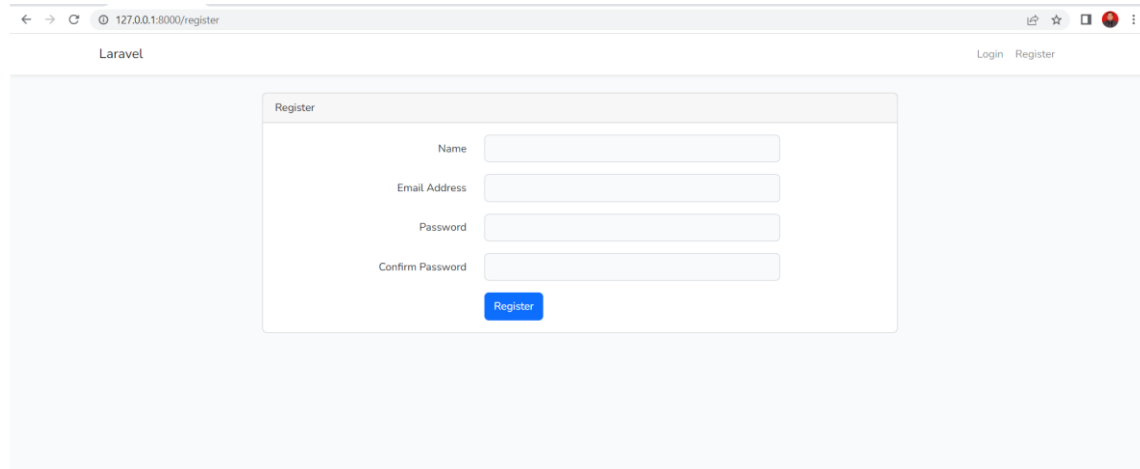
Akses url <http://127.0.0.1:8000/login> pada browser maka akan tampil halaman login seperti berikut.



Gambar Halaman Login Authentication

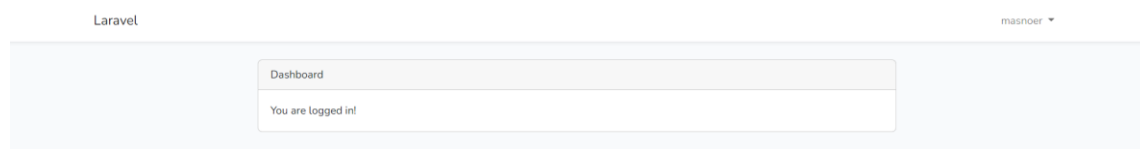
Halaman Register

Akses url <http://127.0.0.1:8000/register> pada browser maka akan tampil halaman register seperti berikut.



Gambar Halaman Register Authentication

Selanjutnya silahkan coba registrasi dan lakukan login ke aplikasi, jika berhasil maka akan tampil seperti gambar berikut.



Gambar Halaman Dashboard

Kesimpulan :

Fitur authentication Laravel akan mengenerate controller, model, view dan route authentication secara otomatis, selain itu juga tampilan view sudah menggunakan framework bootstrap sehingga mempermudah dalam membuat fitur authentication pada aplikasi.

Costum Table Users

Authentication Laravel secara otomatis akan mengenerate table Users yang berisi tentang informasi data user, berikut struktur table users.

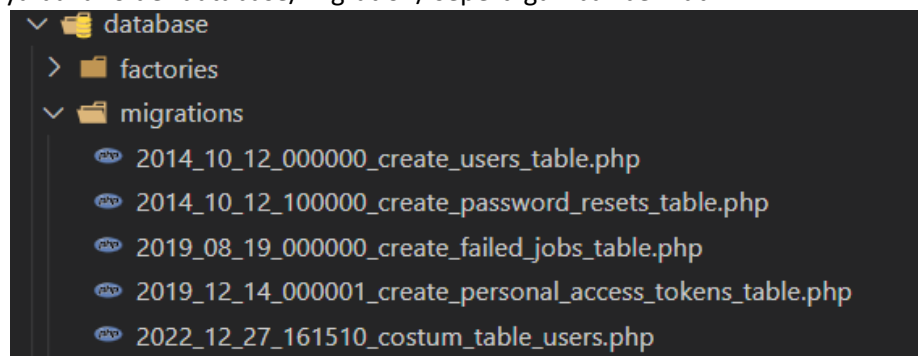
#	Name	Type
1	id 🔑	bigint(20)
2	name	varchar(255)
3	email 🔑	varchar(255)
4	email_verified_at	timestamp
5	password	varchar(255)
6	remember_token	varchar(100)
7	created_at	timestamp
8	updated_at	timestamp

Gambar Struktur Table Users

Dari struktur table users diatas maka perlu ditambahkan beberapa field yaitu username, level dan status, maka perlu membuat sebuah migration untuk menambahkan field-field diatas, ketikkan perintah berikut pada terminal atau cmd.

php artisan make:migration costum_table_users

selanjutnya buka folder database/migration/ seperti gambar berikut.



Gambar Folder Migration

Kemudian bukan file yang diakhiri **costum_table_users.php** dan isikan dengan kode program berikut.

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        //
        Schema::table('users', function (Blueprint $table) {
            $table->string("username")->unique();
            $table->string("level");
            $table->enum("status", ["ACTIVE", "INACTIVE"]);
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn("username");
            $table->dropColumn("level");
            $table->dropColumn("avatar");
            $table->dropColumn("status");
        });
    }
};

```

Kemudian jalankan perintah **php artisan migrate**, jika berhasil maka struktur table users telah berubah sesuai dengan kebutuhan aplikasi.

#	Name	Type
1	id 🔑	bigint(20)
2	name	varchar(255)
3	email 🔑	varchar(255)
4	email_verified_at	timestamp
5	password	varchar(255)
6	remember_token	varchar(100)
7	created_at	timestamp
8	updated_at	timestamp
9	username 🔑	varchar(255)
10	level	varchar(255)
11	status	enum('ACTIVE', 'INACTIVE')

Gambar Costum Struktur Table Users

Seeding User

Setelah berhasil melakukan costum table users selanjutnya membuat user menggunakan fitur seeding pada Laravel, buat seeder dengan nama AdminSeeder dengan perintah :

php artisan make:seeder AdminSeeder

maka secara otomatis file AdminSeeder.php akan dibuat pada folder **database/Seeder**. Kemudian buka file tersebut dan buat akun admin seperti kode program berikut.

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class AdminSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        $admin = new \App\Models\User;
        $admin->username = "admin";
        $admin->name = "Admin Aplikasi";
```

```

$admin->email = "admin@sisfo.com";
$admin->level = json_encode(["ADMIN"]);
$admin->password = \Hash::make("12345678");

$admin->save();

$this->command->info("User Admin berhasil ditambahkan");
}
}

```

Selanjutnya untuk menjalankan seeding dengan cara menjalankan perintah berikut .
php artisan db:seed --class=AdminSeeder jika berhasil maka akan tampil seperti gambar berikut.

```

λ php artisan db:seed --class=AdminSeeder

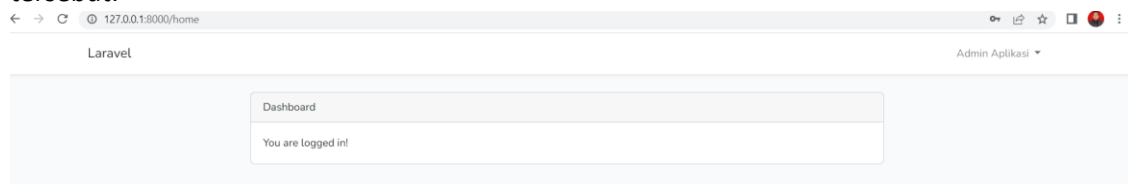
INFO Seeding database.

User Admin berhasil ditambahkan

```

Gambar Seeding Users

Jika sudah berhasil menambahkan user maka dapat login ke aplikasi menggunakan akun tersebut.



Gambar Berhasil Login Menggunakan User Baru

4. Templating atau Layouting

Authentication Laravel secara otomatis mengenerate tampilan dashboard aplikasi, Langkah selanjutnya kita akan konfigurasi tampilan aplikasi sesuai dengan template yang kita pilih, ada studi kasus ini kita akan menggunakan template SB Admin 2 dengan framework bootstrap, silahkan downloah template pada link <https://startbootstrap.com/theme/sb-admin-2> kemudian ekstrak, selanjutnya buat folder pada **public** project Laravel dengan nama **sbadmin** dan copy paste kan seluruh asset template sbadmin kedalam folder tersebut.

Halaman Login Aplikasi

Buka file **app.blade.php** pada folder **view/layouts** kemudian ganti dengan kode program dibawah ini.

```

<!DOCTYPE html>
<html lang="en">

<head>

```



```

<meta charset="utf-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">
<meta name="description" content="">
<meta name="author" content="">

<title>Sisfo - Login</title>

<!-- Custom fonts for this template-->
<link href="{{asset('sbadmin/vendor/fontawesome-
free/css/all.min.css')}}" rel="stylesheet" type="text/css">
<link
    href="https://fonts.googleapis.com/css?family=Nunito:200,200i,300,
300i,400,400i,600,600i,700,700i,800,800i,900,900i"
    rel="stylesheet">

<!-- Custom styles for this template-->
<link href="{{asset('sbadmin/css/sb-admin-2.min.css')}}"
rel="stylesheet">
</head>

<body class="bg-gradient-primary">

    <div class="container">

        <!-- Outer Row -->
        <div class="row justify-content-center">

            <div class="col-xl-6 col-lg-6 col-md-9">

                <div class="card o-hidden border-0 shadow-lg my-5">
                    <div class="card-body p-0">
                        <!-- Nested Row within Card Body -->
                        <div class="row">
                            <!-- <div class="col-lg-6 "></div> -->
                            <div class="col-lg-12 text-center">
                                <div class="p-5">
                                    <div class="text-center">
                                        <h1 class="h4 text-gray-900 mb-
4">Halaman Login</h1>
                                    </div>
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        <form class="user" method="POST"
action="{{ route('login') }}">
            @csrf
            <div class="form-group">
                <input id="email" type="email"
class="form-control form-control-user @error('email') is-invalid
@enderror"
                    name="email" value="{{
old('email') }}" required autocomplete="email" autofocus/>
                    @error('email')
                        <span class="invalid-
feedback" role="alert">
                            <strong>{{ $message
}}</strong>
                        </span>
                    @enderror
                </div>
                <div class="form-group">
                    <input id="password"
type="password" class="form-control form-control-user @error('password')
is-invalid @enderror"
                        name="password" value="{{
old('password') }}" required />
                        @error('password')
                            <span class="invalid-
feedback" role="alert">
                                <strong>{{ $message
}}</strong>
                            </span>
                        @enderror
                    </div>
                    <button type="submit" class="btn
btn-primary btn-user btn-block">
                        login
                    </button>
                </form>
            </div>
        </div>
    </div>
</div>
</div>

```

```

        </div>

    </div>

</div>

<!-- Bootstrap core JavaScript-->
<script
src="{{asset('sbadmin/vendor/jquery/jquery.min.js')}}"></script>
<script src="vendor/bootstrap/js/bootstrap.bundle.min.js"></script>

<!-- Core plugin JavaScript-->
<script src="{{asset('sbadmin/vendor/jquery-
easing/jquery.easing.min.js')}}"></script>

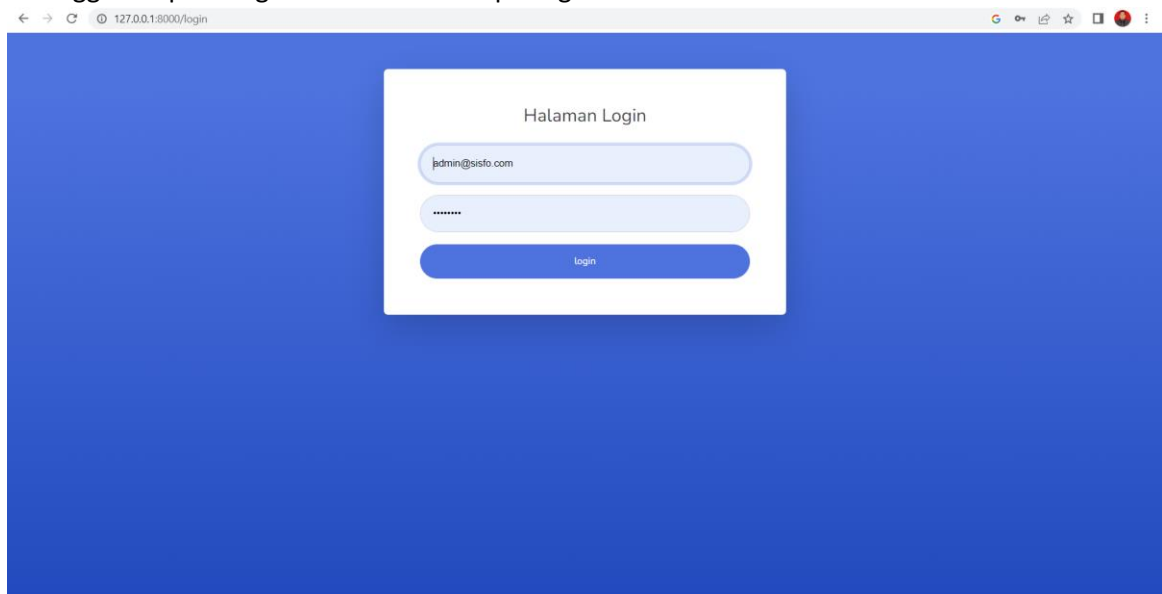
<!-- Custom scripts for all pages-->
<script src="{{asset('sbadmin/js/sb-admin-2.min.js')}}"></script>

</body>

</html>

```

Sehingga tampilan login akan berubah seperti gambar dibawah ini.



Gambar Halaman Login

Layout Global

Selanjutnya membuat layout global yang mana layout ini akan digunakan sebagai layout utama aplikasi dan view-view lain akan mengambil resource disini, karena app.blade.php sudah digunakan untuk view login, maka kita perlu membuat layout baru, silahkan buat file dengan nama **main.blade.php** pada folder **view/layouts** dan isikan dengan kode program berikut ini.

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">
    <meta name="description" content="">
    <meta name="author" content="">

    <title>Sisfo - @yield('judul')</title>

    <!-- Custom fonts for this template-->
    <link href="{{asset('sbadmin/vendor/fontawesome-
free/css/all.min.css')}}" rel="stylesheet" type="text/css">
    <link
        href="https://fonts.googleapis.com/css?family=Nunito:200,200i,300,
300i,400,400i,600,600i,700,700i,800,800i,900,900i"
        rel="stylesheet">

    <!-- Custom styles for this template-->
    <link href="{{asset('sbadmin/css/sb-admin-2.min.css')}}"
rel="stylesheet">

</head>

<body id="page-top">

    <!-- Page Wrapper -->
    <div id="wrapper">

        <!-- Sidebar -->
        @include("layouts.sidebar")
        <!-- End of Sidebar -->

        <!-- Content Wrapper -->
        <div id="content-wrapper" class="d-flex flex-column">
```

```

        <!-- Main Content -->
        <div id="content">

            <!-- Topbar -->
            @include('layouts.topbar')
            <!-- End of Topbar -->

            <!-- Begin Page Content -->
            <div class="container-fluid">

                <!-- Page Heading -->
                <h1 class="h3 mb-4 text-gray-800">@yield("judul")</h1>
                @yield("konten")

            </div>
            <!-- /.container-fluid -->

        </div>
        <!-- End of Main Content -->

        <!-- Footer -->
        <footer class="sticky-footer bg-white">
            <div class="container my-auto">
                <div class="copyright text-center my-auto">
                    <span>Copyright &copy; Sisfo</span>
                </div>
            </div>
        </footer>
        <!-- End of Footer -->

    </div>
    <!-- End of Content Wrapper -->

</div>
<!-- End of Page Wrapper -->

<!-- Scroll to Top Button-->
<a class="scroll-to-top rounded" href="#page-top">
    <i class="fas fa-angle-up"></i>
</a>

<!-- Logout Modal-->
<div class="modal fade" id="logoutModal" tabindex="-1" role="dialog"
aria-labelledby="exampleModalLabel"

```

```

        aria-hidden="true">
        <div class="modal-dialog" role="document">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title" id="exampleModallabel">Yakin
akan keluar aplikasi ?</h5>
                    <button class="close" type="button" data-
dismiss="modal" aria-label="Close">
                        <span aria-hidden="true">x</span>
                    </button>
                </div>
                <div class="modal-body">Silahkan klik tombol logout untuk
keluar aplikasi</div>
                <div class="modal-footer">
                    <button class="btn btn-secondary" type="button" data-
dismiss="modal">Cancel</button>
                    <form action="{{route('logout')}}" method="POST">
                        @csrf
                        <button class="btn btn-primary"
style="cursor:pointer">Logout</button>
                    </form>
                </div>
            </div>
        </div>
    </div>

    <!-- Bootstrap core JavaScript-->
    <script
src="{{asset('sbadmin/vendor/jquery/jquery.min.js')}}"></script>
    <script
src="{{asset('sbadmin/vendor/bootstrap/js/bootstrap.bundle.min.js')}}"></s
cript>

    <!-- Core plugin JavaScript-->
    <script src="{{asset('sbadmin/vendor/jquery-
easing/jquery.easing.min.js')}}"></script>

    <!-- Custom scripts for all pages-->
    <script src="{{asset('sbadmin/js/sb-admin-2.min.js')}}"></script>

</body>

</html>

```

Penjelasan :

Pada layouts **main.blade.php** diatas layout ini akan memanggil **view sidebar.blade.php** dan view **topbar.blade.php** kedalam **layout main.blade.php** dengan menggunakan perintah **@include** sehingga tampilan kedua view tersebut akan tampil pada layout main. Selanjutnya juga akan menampilkan data **judul** dan **konten** dari view yang akan menggunakan layouts **main.blade.php** dengan perintah **@yeild('judul')** dan **@yeild('konten')**.

Sidebar

Buat file pada folder **layouts** dengan nama **sidebar.blade.php** dan isi dengan kode program berikut.

```
<ul class="navbar-nav bg-gradient-primary sidebar sidebar-dark accordion"
id="accordionSidebar">

    <!-- Sidebar - Brand -->
    <a class="sidebar-brand d-flex align-items-center justify-
content-center" href="#">
        <div class="sidebar-brand-icon rotate-n-15">
            <i class="fas fa-laugh-wink"></i>
        </div>
        <div class="sidebar-brand-text mx-3">Sisfo</div>
    </a>

    <!-- Divider -->
    <hr class="sidebar-divider my-0">

    <!-- Nav Item - Dashboard -->
    <li class="nav-item">
        <a class="nav-link" href="index.html">
            <i class="fas fa-fw fa-tachometer-alt"></i>
            <span>Dashboard</span></a>
        </li>

    <!-- Divider -->
    <hr class="sidebar-divider d-none d-md-block">

    <!-- Sidebar Toggler (Sidebar) -->
    <div class="text-center d-none d-md-inline">
        <button class="rounded-circle border-0"
id="sidebarToggle"></button>
    </div>

</ul>
```

Topbar

Buat file pada folder **layouts** dengan nama **topbar.blade.php** dan isi dengan kode program berikut.

```
<nav class="navbar navbar-expand navbar-light bg-white topbar mb-4 static-top shadow">

    <!-- Sidebar Toggle (Topbar) -->
    <button id="sidebarToggleTop" class="btn btn-link d-md-none rounded-circle mr-3">
        <i class="fa fa-bars"></i>
    </button>
    <!-- Topbar Navbar -->
    <ul class="navbar-nav ml-auto">

        <div class="topbar-divider d-none d-sm-block"></div>

        <!-- Nav Item - User Information -->
        <li class="nav-item dropdown no-arrow">
            @if (\Auth::user())
                <a class="nav-link dropdown-toggle" href="#"
id="userDropdown" role="button"
                data-toggle="dropdown" aria-haspopup="true" aria-expanded="false">
                    <span class="mr-2 d-none d-lg-inline text-gray-600 small">{{ \Auth::user()->name }}</span>
                    
                @endif

            <!-- Dropdown - User Information -->
            <div class="dropdown-menu dropdown-menu-right shadow animated-grow-in"
                aria-labelledby="userDropdown">
                <a class="dropdown-item" href="#">
                    <i class="fas fa-user fa-sm fa-fw mr-2 text-gray-400"></i>
                    Profile
                </a>
                <a class="dropdown-item" href="#">
                    <i class="fas fa-cogs fa-sm fa-fw mr-2 text-gray-400"></i>
                    Settings
            </div>
        </li>
    </ul>
</nav>
```



```

        </a>
        <div class="dropdown-divider"></div>
        <a class="dropdown-item" href="#" data-toggle="modal"
data-target="#logoutModal">
            <i class="fas fa-sign-out-alt fa-sm fa-fw mr-2 text-
gray-400"></i>
            Logout
        </a>
    </div>
</li>

</ul>

</nav>

```

Cara penggunaan layouts main.blade.php

Sebagai contoh kita akan menggunakan view [home.blade.php](#), buka file tersebut dan isikan dengan kode program berikut.

```

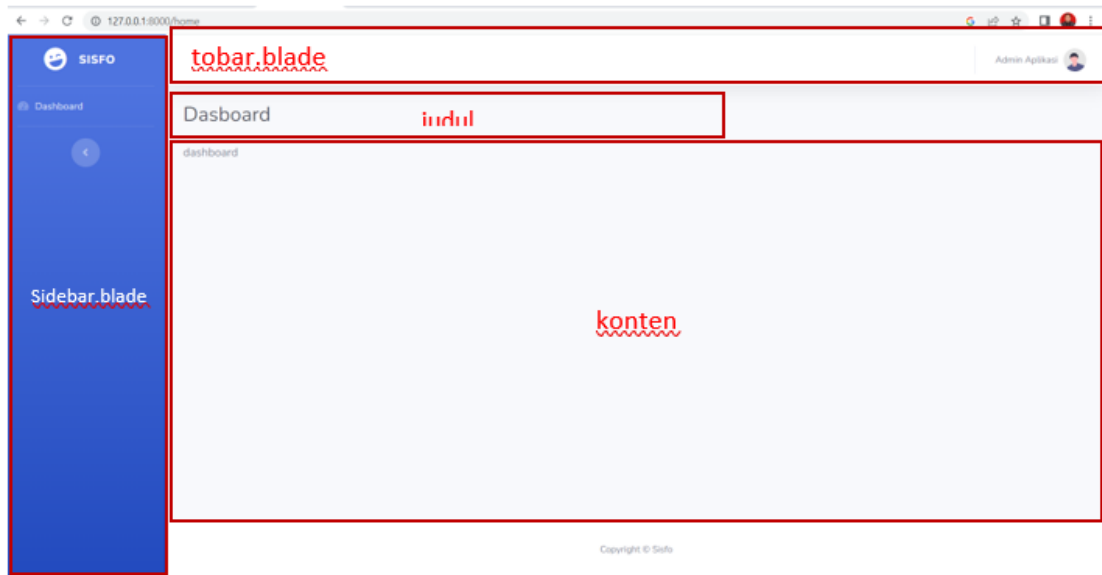
@extends('layouts.main')
@section('judul') Dashboard @endsection
@section('konten')
<P>dashboard</P>
@endsection

```

Penjelasan :

Untuk menggunakan layouts main menggunakan **@extends('layouts.main')** sehingga seluruh tampilan main.blade.php akan ditampilkan pada view home, untuk menampilkan data judul pada layout main dengan perintah **@section('judul')Dashboard @endsection** dan konten dengan cara **@section('judul')Dashboard @endsection**

Jika konfigurasi layout berhasil maka akan tampil seperti gambar berikut.



5. Manajamen Users

Selanjtnya membuat fitur manajemen users, kita akan menggunakan fungsi keseluruhan CRUD maka kita akanmbuat **controller resource** , buka terminal/CMD kemudian ketikkan perintah berikut.

php artisan make:controller UserController –resource

selanjutkan tambahkan kode program berikut pada route web.

```
use App\Http\Controllers\UserController;  
Route::resource('users', UserController::class);
```

Sehingga jika dilihat pemetaan route **php artisan route:list** menjadi seperti gambar di bawah ini.

GET HEAD	users		users.index	> UserController@index
POST	users		users.store	> UserController@store
GET HEAD	users/create		users.create	> UserController@create
GET HEAD	users/{user}		users.show	> UserController@show
PUT PATCH	users/{user}		users.update	> UserController@update
DELETE	users/{user}		users.destroy	> UserController@destroy
GET HEAD	users/{user}/edit		users.edit	> UserController@edit

File UserController Resource

```
<?php
```

```

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Display a listing of the resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function index()
    {
        //
    }

    /**
     * Show the form for creating a new resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function create()
    {
        //
    }

    /**
     * Store a newly created resource in storage.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
        //
    }

    /**
     * Display the specified resource.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function show($id)
    {

```

```

        //
    }

    /**
     * Show the form for editing the specified resource.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function edit($id)
    {
        //
    }

    /**
     * Update the specified resource in storage.
     *
     * @param \Illuminate\Http\Request $request
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function update(Request $request, $id)
    {
        //
    }

    /**
     * Remove the specified resource from storage.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
    public function destroy($id)
    {
        //
    }
}

```

Create Users

Route untuk menampilkan view form tambah data users adalah **users/create** silahkan buka file UserController kemudian pada action create edit kode program menjadi seperti berikut.

```

public function create()
{
    return view('user.create');
}

```

```
}
```

Perintah **return view('user.create')** artinya menampilkan view **create** pada folder **user**, selanjutnya buat folder **user** didalam folder **view**, kemudian tambahkan file **create.blade.php** dan isikan dengan kode program berikut ini.

```
@extends('layouts.main')
@section("judul") Create User @endsection
@section('konten')
<P>Konten</P>
@endsection
```

Selanjutnya tambahkan file css dan javascript select2 kedalam layout **main.blade.php**

```
<link href="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/css/select2.min.css" rel="stylesheet" />
<script src="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/js/select2.min.js"></script>
```

```
<script>
$(document).ready(function() {
    $('.select2-multiple').select2({
        placeholder: "Pilih",
        allowClear: true
    });
});
</script>
```

Selanjutnya membuat form inputan create user, silahkan edit **user/create.blade.php** menjadi seperti kode program berikut.

```
@extends('layouts.main')
@section("judul") Create User @endsection
@section('konten')
<div class="card shadow mb-4">
    <div class="card-header py-3">
    </div>
    <div class="card-body">

        <div class="row">
            <div class="col-lg-9">

                <form method="POST" action="{{ route('users.store') }}">
                    @csrf
                    <div class="form-group row">
```

```

        <label class="col-sm-3 col-form-label text-
center">Nama</label>
        <div class="col-sm-9">
            <input type="text" class="form-control"
id="nama" name="nama">
        </div>
    </div>
    <div class="form-group row">
        <label class="col-sm-3 col-form-label text-
center">Email</label>
        <div class="col-sm-9">
            <input type="email" class="form-control"
id="email" name="email">
        </div>
    </div>
    <div class="form-group row">
        <label class="col-sm-3 col-form-label text-
center">Username</label>
        <div class="col-sm-9">
            <input type="text" class="form-control"
id="username" name="username">
        </div>
    </div>
    <div class="form-group row">
        <label class="col-sm-3 col-form-label text-
center">Password</label>
        <div class="col-sm-2">
            <input type="password" class="form-control"
id="password" name="password">
        </div>
    </div>
    <div class="form-group row">
        <label class="col-sm-3 col-form-label text-
center">Level</label>
        <div class="col-sm-4 mr-2">
            <select class="form-control select2-multiple"
name="level[]" multiple="multiple">
                <option value="ADMIN">ADMIN</option>
                <option value="GURU">GURU</option>
                <option value="STAFF">STAFF</option>
            </select>
        </div>
    </div>
</div>

```

```

        <div class="form-group row">
            <div class="col-sm-10 text-center">
                <button type="reset" class="btn btn-warning btn-sm">Batal</button>
                <button type="submit" class="btn btn-primary btn-sm">Simpan</button>
            </div>
        </div>
    </form>
</div>
</div>
</div>
@endsection

```

Sehingga tampilan form inputan user menjadi seperti gambar dibawah ini.

The screenshot shows a web browser window with the URL 127.0.0.1:8000/users/create. The page has a blue sidebar on the left with a 'Dashboard' link and a back arrow. The main content area is titled 'Create User' and contains a form with the following fields: 'Nama' (text input), 'Email' (text input), 'Username' (text input), 'Password' (password input), and 'Level' (dropdown menu with 'Pilih' selected). Below the form are two buttons: 'Batal' (yellow) and 'Simpan' (blue). The top right corner shows a user profile 'Admin Aplikasi'.

Selanjutnya buka action store pada UserController kemudian isikan kode program berikut.

```

public function store(Request $request)
{
    $user = new \App\Models\User;
    $user->name = $request->get('nama');
}

```

```

        $user->username = $request->get('username');
        $user->email = $request->get('email');
        $user->password = \Hash::make($request->get('password'));
        $user->level = json_encode($request->get('level'));

        $user->save();

        return redirect()->route('users.index')->with('status','user baru
berhasil ditambahkan');
    }

```

Penjelasan :

Form create user menggunakan method **POST** yang akan dikirimkan ke **action store** dengan route **users.store**, selanjutnya pada action store akan ditangkap isi form create user dengan cara **\$request->get('nama input')** kemudian disimpan pada table user dengan cara instance model User dengan cara **\$user = new \App\Models\User** setelah itu assign sesuai dengan nama field. Untuk menyimpan kedalam database User menggunakan perintah **\$user->save**, jika data berhasil disimpan akan di redirect ke route **users.index** dengan membawa session status.

READ / LIST USERS

Selanjutnya setelah fungsi create selesai kita akan menampilkan data user, untuk menampilkan data user kita akan menggunakan method GET yaitu <http://localhost/users> yang mana route ini merupakan action index pada UserController, buka action index kemudian edit menjadi seperti kode program berikut.

```

public function index()
{
    $user = \App\Models\User::all;
    return view('users.index', ['user' => $users]);
}

```

Selanjutnya buat file dengan nama **index.blade.php** pada folder **views/user** dan isi dengan kode program berikut.

```

@extends('layouts.main')
@section("judul") Users @endsection
@section('konten')
@if(session('status'))
    <div class="alert alert-success">
        {{session('status')}}
    </div>
@endif
<div class="card shadow mb-4">

```



```

<div class="card-header py-3">
</div>
<div class="card-body">
    <div class="table-responsive">
        <table class="table table-bordered" id="dataTable" width="100%"
cellspacing="0">
            <thead>
                <tr>
                    <th><b>Name</b></th>
                    <th><b>Username</b></th>
                    <th><b>Email</b></th>
                    <th><b>Action</b></th>
                </tr>
            </thead>
            <tbody>
                @foreach($users as $user)
                    <tr>
                        <td>{{ $user->name }}</td>
                        <td>{{ $user->username }}</td>
                        <td>{{ $user->email }}</td>
                        <td>
                            [action]
                        </td>
                    </tr>
                @endforeach
            </tbody>
        </table>
    </div>
</div>
</div>
@endsection

```

Karena menggunakan datatables tambahkan file css datatable pada bagian head main.blade.php dan file js datatables pada bagian bawah main.blade.php.

```

<link href="{{ asset('sbadmin/vendor/datatables/dataTables.bootstrap4.min.css') }}" rel="stylesheet">
<script src="{{ asset('sbadmin/vendor/datatables/jquery.dataTables.min.js') }}"></script>
<script src="{{ asset('sbadmin/vendor/datatables/dataTables.bootstrap4.min.js') }}"></script>
<script src="{{ asset('sbadmin/js/demo/datatables-demo.js') }}"></script>

```

Selanjutnya menambahkan button untuk membuka form tambah user seperti kode program berikut.

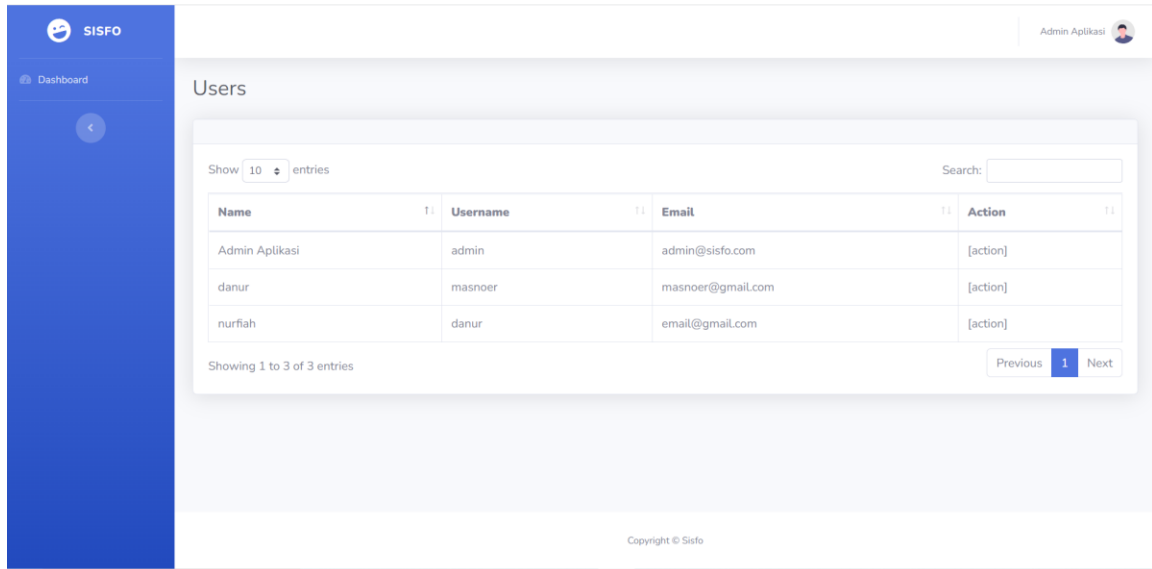
```

<p>
    <a href="{{ route('users.create') }}" class="btn btn-primary btn-sm">Tambah User</a>

```

</p>

Sehingga tampilan list user menjadi seperti gambar dibawah ini.



UPDATE USERS

Selanjutnya untuk melakukan proses update data user terlebih dahulu kita buat tombol action yang mengarahkan **pada route users/edit/{user}/edit** atau action edit pada UserController untuk menampilkan form dan menggunakan **route users/edit/{user}** untuk method PUT. Pertama-tama kita akan membuat tombol edit pada tampilan list user, silahkan buka file user/index.blade.php pada kolom [action] ganti dengan kode berikut ini.

```
<a href="{ route('users.edit', $user->id) }" class="btn btn-sm btn-success">Edit</a>
```

Jika salah satu tombol diatas diklik maka akan menghasilkan string url <http://localhost/users/{user}/edit> yang mana {user} merupakan id dari user yang di klik. Selanjutnya buka action edit pada UserController dan isikan denga kode program berikut.

```
public function edit($id)
{
    $user = \App\Models\User::findOrFail($id);

    return view('user.edit', ['user' => $user]);
}
```

Kode diatas akan mengambil data user berdasarkan id user yang didapatkan dan ditampung pada variable \$user dan dikirimkan ke view user.edit, silahkan buat view didalam folder user/edit.blade.php dan isikan dengan kode program berikut.

```
@extends('layouts.main')
@section("judul") Edit User @endsection
@section('konten')
```

```

<div class="card shadow mb-4">
  <div class="card-header py-3">
  </div>
  <div class="card-body">
    <div class="row">
      <div class="col-lg-9">
        <form method="POST" action="{{ route('users.update',
[$user->id]) }}">
          <input type="hidden" name="_method" value="PUT">
          @csrf
          <div class="form-group row">
            <label class="col-sm-3 col-form-label text-
center">Nama</label>
            <div class="col-sm-9">
              <input type="text" class="form-control"
id="nama" name="nama" value="{{ $user->name }}">
            </div>
          </div>
          <div class="form-group row">
            <label class="col-sm-3 col-form-label text-
center">Email</label>
            <div class="col-sm-9">
              <input type="email" class="form-control"
id="email" name="email" value="{{ $user->email }}">
            </div>
          </div>
          <div class="form-group row">
            <label class="col-sm-3 col-form-label text-
center">Username</label>
            <div class="col-sm-9">
              <input type="text" class="form-control"
id="username" name="username" value="{{ $user->username }}"
              >
            </div>
          </div>
          <div class="form-group row">
            <label class="col-sm-3 col-form-label text-
center">Level</label>
            <div class="col-sm-4 mr-2">
              <select class="form-control select2-multiple"
name="level[]" multiple="multiple">
                <option value="ADMIN" {{in_array("ADMIN",
json_decode($user->level)) ? "selected" : ""}}>ADMIN</option>

```

```

        <option value="GURU" {{in_array("GURU",
json_decode($user->level)) ? "selected" : ""}}>GURU</option>
        <option value="STAFF" {{in_array("STAFF",
json_decode($user->level)) ? "selected" : ""}}>STAFF</option>
    </select>
</div>
</div>
<div class="form-group row">
    <div class="col-sm-10 text-center">
        <a href="{{ route('users.index') }}" class="btn
btn-warning btn-sm">Batal</a>
        <button type="submit" class="btn btn-primary btn-
sm">Simpan</button>
    </div>
</div>
</form>
</div>
</div>
</div>
@endsection

```

Sehingga tampilan form edit user menjadi seperti gambar berikut ini.

The screenshot shows the 'Edit User' form in the SISFO application. The form is displayed in a modal window with a light blue header and a white body. It contains four input fields: 'Nama' (Admin Aplikasi), 'Email' (admin@sisfo.com), 'Username' (admin), and 'Level' (a dropdown menu showing 'ADMIN'). Below the fields are two buttons: 'Batal' (yellow) and 'Simpan' (blue). The background shows the SISFO dashboard with a blue sidebar and a top navigation bar.

Selanjutnya buka action update pada UserController kemudian isikan dengan kode program berikut ini.

```

public function update(Request $request, $id)

```

```

{
    $user = \App\Models\User::findOrFail($id);

    $user->name = $request->get('nama');
    $user->level = json_encode($request->get('level'));

    $user->save();

    return redirect()->route('users.index', [$id])->with('status',
    'User berhasil diubah');
}

```

Selanjutnya kita akan mencoba melakukan perubahan data user seperti gambar dibawah ini.

Edit User

Nama	Admin Aplikasi edit
Email	admin@sisfo.com
Username	admin
Level	ADMIN

Sesuai gambar diatas kita akan mengubah nama user menjadi fulan bin fulan seperti gambar berikut.

Edit User

Nama	Fulan bin Fulan
Email	admin@sisfo.com
Username	admin
Level	ADMIN

Setelah diganti nama user tersebut klik simpan, jika berhasil maka akan diredirect ke halaman users dengan menampilkan alert success seperti gambar berikut.

Fulan bin Fulan

Users

Tambah User

User berhasil diubah

Show 10 entries

Search:

Name	Username	Email	Action
danur	masnoer	masnoer@gmail.com	Edit
Fulan bin Fulan	admin	admin@sisfo.com	Edit
nurfiah	danur	email@gmail.com	Edit

Showing 1 to 3 of 3 entries

Previous

1

Next

DELETE USERS

Tambahkan tombol atau link pada list user, silahkan buka view user/index.blade.php dan tambahkan kode program berikut dibawah link edit.

```

<form
    onsubmit="return confirm('Hapus data user ?');"
    class="d-inline"
    action="{{route('users.destroy', [$user-
>id])}}"

    method="POST">
        @csrf
        <input
            type="hidden"
            name="_method"
            value="DELETE">

        <input
            type="submit"
            value="Hapus"
            class="btn btn-danger btn-sm">

    </form>

```

Selanjutnya buka action destroy pada UserController dan isikan dengan kode program berikut ini.

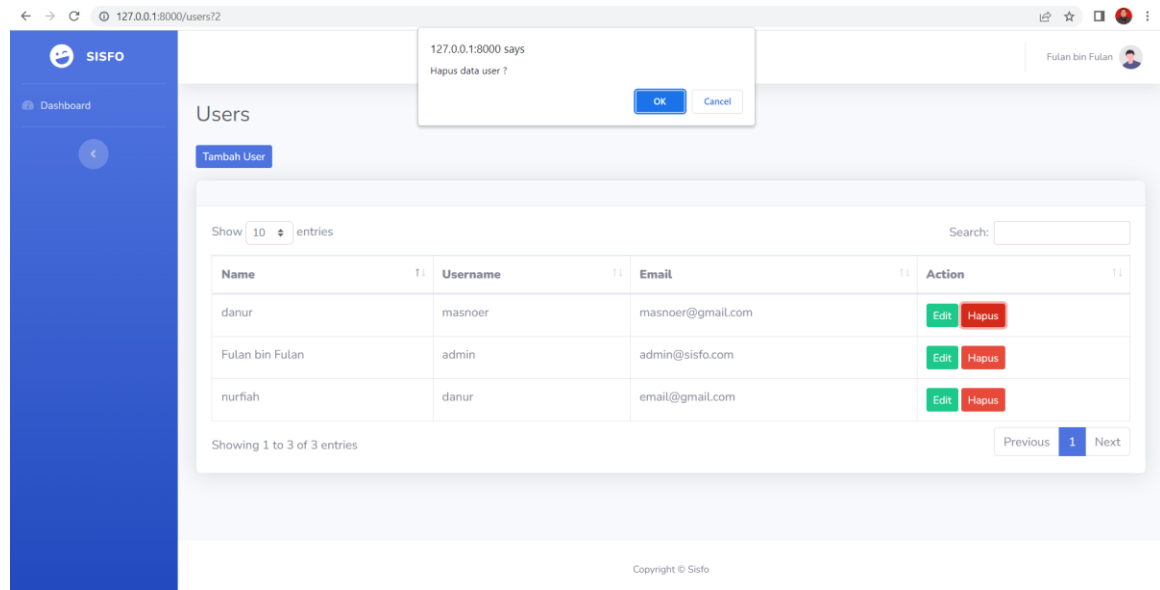
```

public function destroy($id)
{
    $user = \App\Models\User::findOrFail($id);
    $user->delete();
}

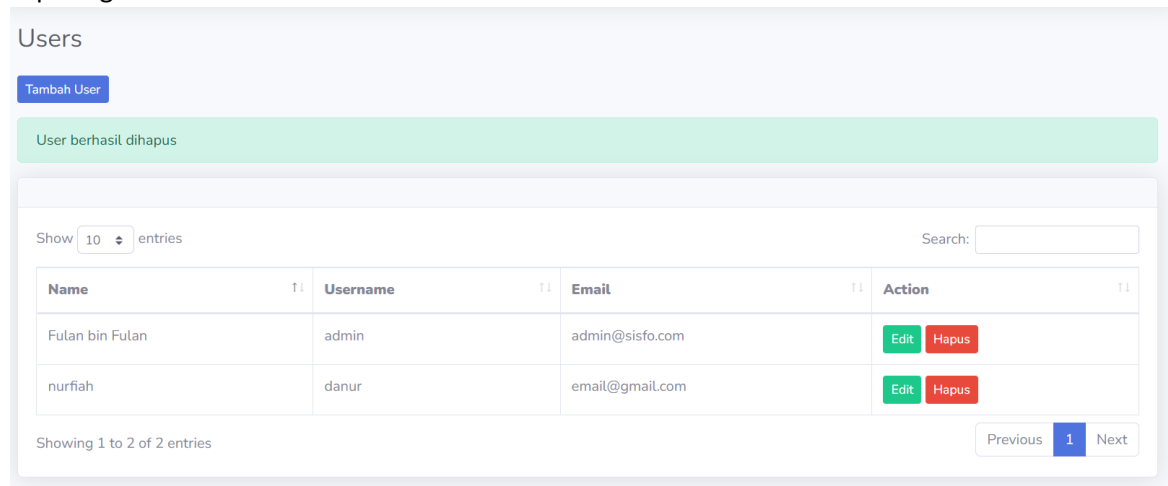
```

```
return redirect()->route('users.index')->with('status', 'User
berhasil dihapus');
}
```

Ketika diklik salah satu tombol hapus maka akan muncul pop up dialog peringatan apakah data akan dihapus seperti gambar dibawah ini.



Jika kita tekan oke maka data akan dihapus, jika data berhasil dihapus maka akan muncul pesan seperti gambar berikut.

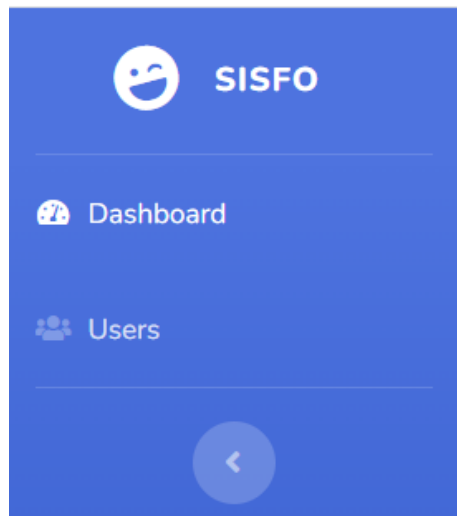


MENAMBAHKAN MENU USERS

Silahkan buka view Layouts/sidebar.blade.php kemudian tambahkan kode program berikut.

```
<li class="nav-item">
    <a class="nav-link" href="{{ route('users.index') }}">
        <i class="fas fa-fw fa-users"></i>
        <span>Users</span></a>
</li>
```

Sehingga pada aplikasi akan muncul menu users seperti gambar berikut ini.



6.